# Ancient Greek Text Classification with BERT & Neural Networks

This project implements deep learning models to classify Ancient Greek inscriptions based on their textual content and metadata. The system leverages **BERT** (Bidirectional Encoder Representations from Transformers) for text tokenization and **PyTorch neural networks** for classification, achieving high accuracy in predicting regional origins and date ranges of ancient texts.

## Dataset

The model uses the **IPhi2802** dataset containing Ancient Greek inscriptions with the following features:

| Column | Description |
|---|---|
| `text` | The inscription text in Ancient Greek |
| `region_main` | Primary region of origin |
| `region_sub` | Sub-region of origin |
| `date_min` | Minimum estimated date |
| `date_max` | Maximum estimated date |
| `metadata`, `id`, `date_str`, `date_circa` | Additional metadata (dropped) |

## Project Structure

```
├── Models/
│   ├── __init__.py
│   ├── Errors.py                  # Custom exception classes
│   ├── PyTorch_NN.py              # PyTorch neural network implementation
with cross-validation
│   ├── FileManagement.py          # File I/O operations (CSV, JSON, exports)
│   ├── Models/DataManagement.py    # Data preprocessing and cleaning utilities
├── BertModel.ipynb        # BERT tokenization and model training notebook
├── main.ipynb             # Main training notebook with TF-IDF vectorization
├── requirements.txt       # Python dependencies
├── exports/               # Training logs and results
│   ├── images/            # Training/validation loss plots
│   ├── *.txt              # Per-fold training metrics
└── data/                  # Dataset directory (not included)
    ├── iphi2802.csv       # Main dataset
    └── greek_stopwords.json # Greek stop words list
```

## Preprocessing Pipeline

The data undergoes several cleaning steps:

1. **Column removal** - Drop non-informative columns (`metadata`, `id`, `region_main`, `region_sub`, `date_str`, `date_circa`)
2. **Text cleaning**:
    - Remove whitespace and special characters (`[]`, `-`, `.`)
    - Convert to lowercase
    - Remove single-character words
    - Remove Greek stop words
    - Filter out words with frequency < 5
3. **Empty cell handling** - Remove rows with empty text after cleaning

## Feature Extraction

Two text vectorization methods are implemented:

### TF-IDF Vectorization (`main.ipynb`)

- Converts text to TF-IDF feature matrix
- Combined with metadata features (`region_main_id`, `region_sub_id`, `date_min`, `date_max`)

### BERT Tokenization (`BertModel.ipynb`)

- Uses `pranaydeeps/Ancient-Greek-BERT` pretrained model
- Tokenizes text with maximum length of 4469 tokens
- Combines token embeddings with metadata features

## Model Architecture

The project implements several neural network architectures using PyTorch:

### Base Model

```
Input (1899 features) → Dense(3560) → ReLU → Dense(2) → Sigmoid
```

### Multi-layer Model

```
Input → Dense(3560) → ReLU → Dense(1780) → ReLU → Dense(2) → Sigmoid
```

### Multi-layer with Dropout

```
Input → Dropout(R_IN) → Dense(3560) → ReLU → Dropout(R_H) →
Dense(1780) → ReLU → Dropout(R_H) → Dense(2) → Sigmoid
```

### Hyperparameter Configurations

- **Input nodes**: 1899 (after TF-IDF)
- **Hidden layer nodes**: 3560 (formula: `2 * INPUT_NODES - INPUT_NODES/8`)
- **Output nodes**: 2 (binary classification)
- **Epochs**: 200
- **Batch size**: 16
- **Learning rate**: 0.001 - 0.1 (tested)
- **Momentum**: 0.2 - 0.6 (tested)
- **Dropout rates**: R_IN = 0.5/0.8, R_H = 0.2/0.5
- **Optimizer**: Adam / SGD
- **Loss function**: MSELoss (for date regression) / CrossEntropyLoss (for classification)

## Training Methodology

- **Cross-validation**: 5-fold K-Fold with shuffling
- **Early stopping**: Configurable patience (default: 5 epochs)
- **Device support**: Automatically uses CUDA (NVIDIA), MPS (Apple Silicon), or CPU
- **Metrics tracked per fold**:
  - Validation Loss
  - RMSE (Root Mean Square Error)

## Results

### Best Performing Configuration

#### Multi-layer Perceptron with SGD optimizer

- Learning rate: 0.001
- Momentum: 0.2
- Hidden nodes: 3560
- **Final RMSE: 0.03786**

### Comparative Results

| Configuration | Learning Rate | Momentum | Dropout | Validation Loss | RMSE |
|---|---|---|---|---|---|
| MLP (SGD) | 0.001 | 0.2 | None | 0.00143 | **0.03786** |
| MLP (Adam) | 0.001 | 0.2 | None | 0.00207 | 0.04551 |
| MLP (SGD) | 0.001 | 0.6 | None | 0.00162 | 0.05334 |
| MLP w/ Dropout | 0.001 | 0.2 | R_IN=0.5, R_H=0.5 | 0.00712 | 0.08436 |
| MLP (SGD) | 0.05 | 0.6 | None | 0.13363 | 0.36599 |
| MLP (SGD) | 0.1 | 0.6 | None | 0.13640 | 0.38386 |

Hidden Layer Size Comparison (200 epochs, batch=16)

| Hidden Nodes | Formula | RMSE |
|---|---|---|
| 633 | `(INPUT + OUTPUT)/3` | 0.14867 |
| 1268 | `(INPUT * 2/3) + OUTPUT` | 0.10822 |
| 3560 | `2 * INPUT - INPUT/8` | **0.08954** |

Key Findings

1. **Lower learning rate (0.001) significantly outperforms higher rates**
2. **SGD with momentum 0.2 outperforms SGD with momentum 0.6**
3. **Larger hidden layer (3560) yields best results**
4. **Dropout may not be beneficial for this dataset** (RMSE increased from 0.03786 to 0.08436)
5. **Adam performs comparably but slightly worse than SGD** (0.04551 vs 0.03786)

## Usage

Installation

```
pip install -r requirements.txt
```

Download NLTK Data (for tokenization)

```
import nltk
nltk.download('punkt')
```

Run TF-IDF Model

```
jupyter notebook main.ipynb
```

Run BERT Model

```
jupyter notebook BertModel.ipynb
```

Custom Training Configuration

```
from Models import PyTorch_NN
```

```
model = PyTorch_NN.PyTorchModel(
    hidden_layer_nodes=3560,
    epochs=200,
    batch_size=16,
    X=X, y=y,
    momentum=0.2,
    learning_rate=0.001
)
model.create_default_model()
model.train_test(with_early_stopping=False)
```

## Custom Loss Function

The project includes a custom **BoundaryDeviationLoss** class that penalizes predictions falling outside the target range (useful for date prediction tasks where the true value lies between `date_min` and `date_max`).

## Output Files

Training results are automatically exported to `exports/` directory:

- `{epochs}_{batch_size}_{hidden_nodes}_{lr}_{momentum}.txt` - Per-fold metrics
- `foo.png` - Training/validation loss curves

## Dependencies

- Python 3.10+
- PyTorch 2.2.0
- scikit-learn 1.4.2
- pandas 2.2.2
- numpy 1.26.4
- matplotlib 3.8.4
- nltk 3.8.1
- transformers (for BERT model)

## Notes

- The dataset file `iphi2802.csv` must be placed in the `data/` directory
- Greek stop words should be in `data/greek_stopwords.json`
- For BERT model, the Ancient-Greek-BERT model is downloaded automatically via HuggingFace
- Decimal values in the dataset are handled automatically
- The project automatically detects and uses GPU acceleration if available

## License

This project is for educational and research purposes. The IPhi2802 dataset and Ancient-Greek-BERT model have their own usage terms - please cite appropriately.

## Acknowledgments

- **IPhi2802** dataset for Ancient Greek inscriptions
- **Pranaydeep Singh** for the Ancient-Greek-BERT model
- PyTorch and HuggingFace teams for their frameworks